Programação funcional

Aula 03 - Conceitos básicos*

*Baseado no material disponível em https://haskell.pesquisa.ufabc.edu.br/haskell

Linguagem Haskell

- Características:
  - Puramente funcional
  - Códigos concisos e declarativos
  - Imutabilidade
  - Funções recursivas

Linguagem Haskell

- Características:
  - Funções de alta ordem
    - podem receber funções como parâmetro
  - Tipos polimórficos
    - funções que funcionam para classes de tipos
  - Avaliação preguiçosa
    - resultados computados apenas quando utilizados

Olá, mundo!

```haskell
module Main where   -- indica que é o módulo principal

main :: IO ()
main = do                       -- início da função principal
  putStrLn "hello world"        -- imprime hello world
```

Funções

Aplicação de funções:

- Tem a maior precedência

```
f a b       -- f(a,b)
f a b + c*d -- f(a,b) + c*d
```

Funções na matemática *vs* Haskell

Matemática	Haskell
$f(x)$	`f x`
$f(x, y)$	`f x y`
$f(g(x))$	`f (g x)`
$f(x, g(y))$	`f x (g y)`
$f(x)g(y)$	`f x * g y`

Convenções

- Nomes de funções e argumentos
  - começar com letra minúscula
  - seguida por 0 ou mais letras, maiúsculas ou minúsculas, dígitos, underscore, e aspas simples:
- Nomes reservados
  - case, class, data, default, deriving do, else, foreign, if, import, in, infix, infixl, infixr, instance, let module, newtype, of, then, type, where

Convenções

- Listas
  - Nomeadas acrescentando o caractere ’s’ ao nome do que ela representa
    - Uma lista de números n é nomeada ns
    - Uma lista de variáveis x se torna xs
    - Uma lista de listas de caracteres tem o nome css.

Convenções

- Layout
  - Similar ao Python (indentação)
  - Tab *vs* espaço
    - A definição de tabulação varia de editor para editor.
    - A tabulação varia de usuário para usuário.
    - Como é importante no Haskell, usem espaços

Convenções

- Comentários
  - Em uma linha:
    - Demarcados pela sequência **--**
  - Em múltiplas linhas:
    - Demarcados por **{-** e **-}**

```
-- função que dobra o valor de x
dobra x = x + x

{-
dobra recebe uma variável numérica
e retorna seu valor em dobro.
-}
```

Tipos de dados

Tipos de dados

- Começam com letras maiúsculas
- Notação para definição de tipo:
  - **v :: T**
  - Significa que **v** é do tipo **T**

Tipos básicos

- Bool
  - Valores True e False.
  - Operadores: **&&** (e), **||** (ou) e **not**
- Char
  - Caracteres no sistema Unicode.
  - Especificados com aspas simples: 'a'
- String
  - Sequências de caracteres delimitados por aspas duplas: "Olá Mundo"

Tipos básicos

- Int
  - inteiros com precisão fixa em 64 bits. Representa os valores numéricos de -2^{63} a $2^{63}-1$
- Integer
  - inteiros de precisão arbitrária. Representa valores inteiros de qualquer precisão, a memória é o limite. Mais lento do que operações com Int.
- Float
  - valores em ponto-flutuante de precisão simples. Permite representar números com um total de 7 dígitos, em média.
- Double
  - valores em ponto-flutuante de precisão dupla. Permite representar números com quase 16 dígitos, em média

Listas

- Sequências de elementos do mesmo tipo agrupados por colchetes e separados por vírgula
  - [1,2,3,4]
  - [12]
- Tipo de uma lista de elementos do tipo T é [T]
  - [1,2,3,4] :: [Int]
  - [False, True, True] :: [Bool]
  - ['o', 'l', 'a'] :: [Char]

Listas

- Listas de listas:
  - [[1,2,3], [4,5]] :: [[Int]]
  - [['o','l','a'], ['m','u','n','d','o']] :: [[Char]]
- Observações:
  - O tipo da lista não especifica seu tamanho
  - Não existem limitações quanto ao tipo da lista
  - Não existem limitações quanto ao tamanho da lista

Tuplas

- Sequências finitas de componentes, contendo zero ou mais tipos diferentes
  - (True, False) :: (Bool, Bool)
  - (1.0, "Sim", False) :: (Double, String, Bool)
- Observações:
  - O tipo da tupla específica seu tamanho
  - Não existem limitações dos tipos associados a tupla
  - Tuplas devem ter um tamanho finito

Funções

- Mapas de argumentos de um tipo para resultados em outro tipo
- O tipo de uma função é escrito como **T1 -> T2**
  - mapa do tipo T1 para o tipo T2
- Exemplos:
  - **not :: Bool -> Bool**
  - **even :: Int -> Bool**

Funções

- Múltiplos argumentos
  - Separar por **->** onde o último é o tipo do retorno

```haskell
soma :: Int -> Int -> Int
soma x y = x + y

mult :: Int -> Int -> Int -> Int
mult x y z = x * y * z
```

Referências

- Material do prof. Emilio Francesquini
  - https://haskell.pesquisa.ufabc.edu.br/haskell